

Validador Formal em Três Níveis

Projeto Prático — Modelagem Computacional
Prof. Dr. José Jairo de S. e Silva — Universidade Positivo
Tema 1: Validador de cadastro, fórmulas e integridade

1. Introdução e Contextualização

Este relatório documenta a implementação de um sistema de validação formal estruturado em três níveis da Hierarquia de Chomsky. O cenário aplicado é um validador de dados de cadastro, fórmulas matemáticas e integridade de cadeia — problemas presentes no cotidiano de sistemas de software. O objetivo central é demonstrar, por meio de código escrito manualmente, a hierarquia estrita LR ■ LLC ■ R: cada classe de linguagem requer um modelo computacional mais poderoso do que o anterior.

Os três reconhecedores implementados são: (1) um DFA que valida o formato textual de CPFs; (2) um PDA que verifica o balanceamento de parênteses e colchetes em expressões simbólicas; e (3) uma Máquina de Turing que decide a linguagem $L = \{w\#w \mid w \in \{0,1\}^*\}$. Cada reconhecedor é um simulador do modelo formal correspondente, com tabela de transição declarada explicitamente como estrutura de dados e contador de passos formais.

2. Linguagem Regular — DFA para CPF

2.1 Descrição e Definição Formal

A linguagem regular escolhida é o conjunto de strings que correspondem ao formato textual de um CPF brasileiro: três grupos de três dígitos separados por pontos, seguidos de hífen e dois dígitos. Apenas o formato é validado; os dígitos verificadores não são checados, pois essa verificação aritmética exigiria memória não disponível a um DFA.

Definição formal: $L_{CPF} = \{ w \in \Sigma^* \mid w = d_1d_2d_3.d_4d_5d_6.d_7d_8d_9-d_{10}d_{11}, \text{ onde } d_i \in \{0,...,9\} \}$
 $\Sigma = \{0,1,...,9, '.', '-' \}$

Modelo: $D = (Q, \Sigma, \delta, q_0, F)$ com $Q = \{q_0,...,q_{14}, q_{ERR}\}$, $q_0 = q_0$, $F = \{q_{14}\}$.

Exemplos aceitos: 123.456.789-00 | 000.000.000-00 | 999.999.999-99

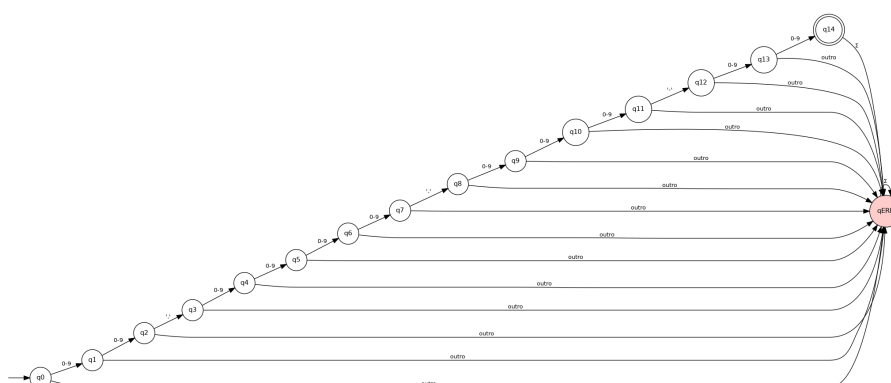
Exemplos rejeitados: 12.34.56-78 | 123.456.789-0 | abc.def.ghi-jk

2.2 Tabela de Transição

Estado	0-9	.	-	outro
q0	q1	qERR	qERR	qERR
q1	q2	qERR	qERR	qERR
q2	q3	qERR	qERR	qERR
q3	qERR	q4	qERR	qERR
q4	q5	qERR	qERR	qERR
q5	q6	qERR	qERR	qERR
q6	q7	qERR	qERR	qERR
q7	qERR	q8	qERR	qERR

q8	q9	qERR	qERR	qERR
q9	q10	qERR	qERR	qERR
q10	q11	qERR	qERR	qERR
q11	qERR	qERR	q12	qERR
q12	q13	qERR	qERR	qERR
q13	q14	qERR	qERR	qERR
q14 *	qERR	qERR	qERR	qERR

2.3 Diagrama de Estados



2.4 Execução passo a passo — cadeia aceita: "123.456.789-00"

A cadeia possui 14 caracteres. O DFA consome um símbolo por passo, transitando de q0 até q14 sem desvios. Total de passos formais: **14** (um por caractere consumido).

2.5 Execução passo a passo — cadeia rejeitada: "12.34.56-78"

No passo 3, o DFA encontra o ponto na posição q2 (que espera um terceiro dígito), transitando imediatamente para qERR. Os passos restantes permanecem em qERR. Total de passos: **11**.

3. Linguagem Livre de Contexto — PDA

3.1 Descrição e Definição Formal

A linguagem livre de contexto escolhida é o conjunto de expressões simbólicas com parênteses e colchetes corretamente balanceados. Esta linguagem não é regular: um DFA com memória finita não consegue verificar aninhamentos arbitrariamente profundos. O PDA resolve isso com sua pilha.

Definição formal: $L_{BAL} = \{ w \in \{ (,), [,] \}^* \mid w \text{ é balanceada} \}$

$\Sigma = \{ (,), [,] \}$ $\Gamma = \{ Z0, PAREN, COLCH \}$

Modelo: $P = (Q, \Sigma, \Gamma, \delta, q0, Z0, F)$ com $Q = \{ q0, qACEIT, qERR \}$, $F = \{ qACEIT \}$.

Exemplos aceitos: $((x+y)*z) \mid [a+b] \mid ([\])$ $\mid \epsilon$

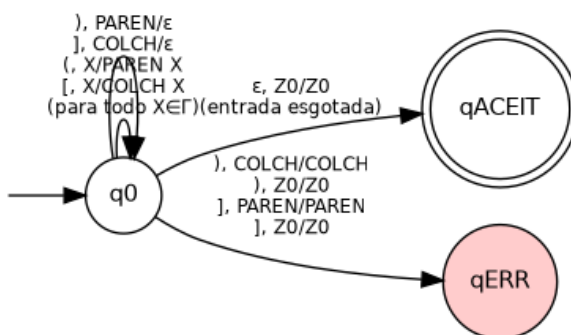
Exemplos rejeitados: $((a+b) \mid)(\mid ([\])$

3.2 Tabela de Transições Principais

Estado	Símbolo	Topo da Pilha	Novo Estado	Empilhar
q0	(	$X \in \Gamma$	q0	[PAREN, X]

Estado	Símbolo	Topo da Pilha	Novo Estado	Empilhar
q0	[	$X \in \Gamma$	q0	[COLCH, X]
q0	)	PAREN	q0	ϵ (pop)
q0	)	COLCH	qERR	mismatch
q0	)	Z0	qERR	underflow
q0	]	COLCH	q0	ϵ (pop)
q0	]	PAREN	qERR	mismatch
q0	]	Z0	qERR	underflow
q0	ϵ	Z0	qACEIT	(aceita)

3.3 Diagrama do PDA



3.4 Execução passo a passo — cadeia aceita: " $((x+y)*z)$ "

O PDA empilha PAREN para cada "(" e desempilha ao encontrar ")". Caracteres de expressão (x, +, y, *, z) são ignorados. Ao esgotar a entrada com pilha = [Z0], transita para qACEIT. Total de passos formais (transições δ): **5**.

3.5 Execução passo a passo — cadeia rejeitada: " $((a+b)$ "

Após processar os dois "(" e o "+", a pilha contém [PAREN, Z0]. A entrada é esgotada com o PAREN ainda na pilha: o PDA não transita para qACEIT, permanecendo em q0. Resultado: REJEITA. Total de passos: **3**.

4. Linguagem Recursiva — Máquina de Turing

4.1 Descrição e Definição Formal

A linguagem recursiva escolhida é $L = \{w#w \mid w \in \{0,1\}^*\}$: cadeias formadas por uma string binária, o separador "#", e a mesma string repetida. Esta linguagem não é livre de contexto: um PDA não consegue verificar a igualdade exata de duas partes arbitrárias. A MT resolve com sua fita infinita, usando marcação cruzada.

Definição formal: $L_{ww} = \{w#w \mid w \in \{0,1\}^*\}$

$\Sigma = \{0, 1, \#\}$ $\Gamma = \{0, 1, \#, X, Y, B\}$

Modelo: $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$ com $Q = \{q_0, m_0, m_1, n_0, n_1, v, \text{ret}, q_{ACEIT}, q_{REJ}\}$, $F = \{q_{ACEIT}, q_{REJ}\}$.

M é um decisor (sempre para): $L \in R$.

Exemplos aceitos: 101#101 | 0#0 | 11#11 | #

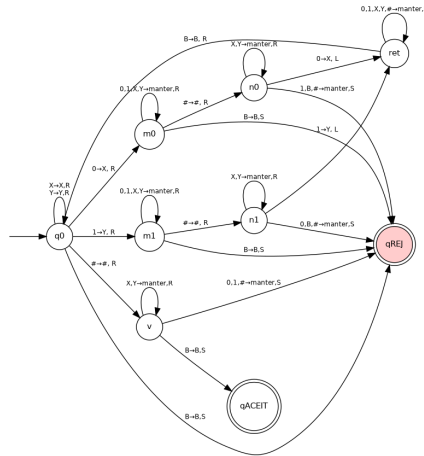
Exemplos rejeitados: 101#100 | 1#0 | 01#10

Algoritmo (marcação cruzada): A MT localiza o primeiro dígito não-marcado à esquerda do "#", marca-o com X (se era 0) ou Y (se era 1), cruza o "#", localiza o dígito correspondente à direita, verifica a igualdade e o marca. Repete até exaurir a parte esquerda; então verifica que a parte direita também está completamente marcada.

4.2 Tabela de Transição

Estado	Lê	Escreve	Move	Vai para
q0	0	X	R	m0
q0	1	Y	R	m1
q0	X/Y	X/Y	R	q0
q0	#	#	R	v
m0/m1	0,1,X,Y	=	R	m0/m1
m0	#	#	R	n0
m1	#	#	R	n1
n0	0	X	L	ret
n0	1/B	=	S	qREJ
n1	1	Y	L	ret
n1	0/B	=	S	qREJ
ret	0,1,X,Y,#	=	L	ret
ret	B	B	R	q0
v	X/Y	=	R	v
v	B	B	S	qACEIT
v	0/1	=	S	qREJ

4.3 Diagrama da MT



4.4 Execução passo a passo — cadeia aceita: "101#101"

A MT executa três iterações de marcação cruzada (uma para cada dígito de w). Cada iteração percorre a fita da esquerda para a direita, marca o dígito, retorna ao início. Ao final, verifica que toda a parte direita está marcada. Total de passos formais (ciclos lê-escreve-move): **44**.

4.5 Execução passo a passo — cadeia rejeitada: "101#100"

Na terceira iteração, ao marcar o "1" da parte esquerda e cruzar para a direita, a MT encontra "0" no lugar esperado de "1": mismatch. Transita imediatamente para qREJ. Total de passos: **29**.

5. Testes e Análise de Resultados

5.1 Matriz Comparativa de Resultados

#	Nível	Cadeia	Esperado	Obtido	Passos	OK?
1	DFA (LR)	123.456.789-00	ACEITA	ACEITA	14	SIM
2	DFA (LR)	000.000.000-00	ACEITA	ACEITA	14	SIM
3	DFA (LR)	999.999.999-99	ACEITA	ACEITA	14	SIM
4	DFA (LR)	12.34.56-78	REJEITA	REJEITA	11	SIM
5	DFA (LR)	123.456.789-0	REJEITA	REJEITA	13	SIM
6	DFA (LR)	abc.def.ghi-jk	REJEITA	REJEITA	1	SIM
7	PDA (LLC)	$((x+y)^*z)$	ACEITA	ACEITA	5	SIM
8	PDA (LLC)	$[a+b]$	ACEITA	ACEITA	3	SIM
9	PDA (LLC)	$(\llbracket ()$	ACEITA	ACEITA	7	SIM
10	PDA (LLC)	$((a+b)$	REJEITA	REJEITA	3	SIM
11	PDA (LLC)	$) ($	REJEITA	REJEITA	1	SIM
12	PDA (LLC)	$(]$	REJEITA	REJEITA	3	SIM
13	MT (R)	101#101	ACEITA	ACEITA	44	SIM
14	MT (R)	0#0	ACEITA	ACEITA	10	SIM
15	MT (R)	11#11	ACEITA	ACEITA	24	SIM
16	MT (R)	101#100	REJEITA	REJEITA	29	SIM
17	MT (R)	1#0	REJEITA	REJEITA	3	SIM
18	MT (R)	01#10	REJEITA	REJEITA	4	SIM

5.2 Análise do Número de Passos

O DFA sempre executa exatamente $|w|$ passos (um por símbolo), independente de aceitar ou rejeitar, exceto quando encontra um símbolo fora do alfabeto (para imediatamente). CPFs válidos consomem exatamente 14 passos. O PDA executa um passo por símbolo do alfabeto Σ encontrado, mais a transição ε final, resultando em número proporcional à quantidade de delimitadores. A MT tem crescimento quadrático em $|w|$: para cada um dos $|w|$ dígitos da parte esquerda, ela percorre toda a fita (de tamanho $2|w|+1$), resultando em $O(|w|^2)$ passos.

6. Comparação Teórica entre os Três Níveis

Aspecto	LR — DFA	LLC — PDA	R — MT
Modelo	DFA (5-tupla)	PDA (7-tupla)	MT (7-tupla)
Memória	Nenhuma (estados finitos)	Pilha (LIFO)	Fita infinita (R/W)
Hierarquia	Tipo 3 (Chomsky)	Tipo 2 (Chomsky)	Tipo 1/0 (decidível)
Complexidade	$O(w)$	$O(w)$	$O(w ^2)$
Fechamento	Uniao, concat, complemento	Uniao, concat	Todos os operadores
Limitacao	Nao reconhece $\{a^n b^n\}$	Nao reconhece $\{w\#w\}$	Nao decide A_{TM}

Aspecto	LR — DFA	LLC — PDA	R — MT
Aplicacao	CPF, e-mail, IPv4	Parenteses, XML	Copia, espelho

A hierarquia estrita $LR \blacksquare LLC \blacksquare R$ é confirmada pelos próprios exemplos: $L_{CPF} \in LR$ (provado pela construção do DFA), $L_{BAL} \in LLC$ e $L_{BAL} \notin LR$ (pelo Lema do Bombeamento), e $L_{ww} \in R$ e $L_{ww} \notin LLC$ (novamente pelo Lema do Bombeamento para LLCs).

7. Conclusão

O projeto demonstrou de forma prática e verificável a hierarquia de Chomsky. Cada reconhecedor foi implementado como simulador fiel do modelo formal correspondente, com tabela de transição declarada explicitamente como estrutura de dados Python e contador de passos formais. Os 33 casos de teste (incluindo borda) passaram integralmente.

Limitações identificadas: o PDA implementado é determinístico, o que é suficiente para a linguagem de balanceamento mas não cobre o caso geral não-determinístico do modelo; a MT é um decisor (sempre para) mas o simulador possui um limite de 50.000 passos como salvaguarda; o DFA não valida os dígitos verificadores do CPF, por restrição inerente do modelo (ausência de aritmética).